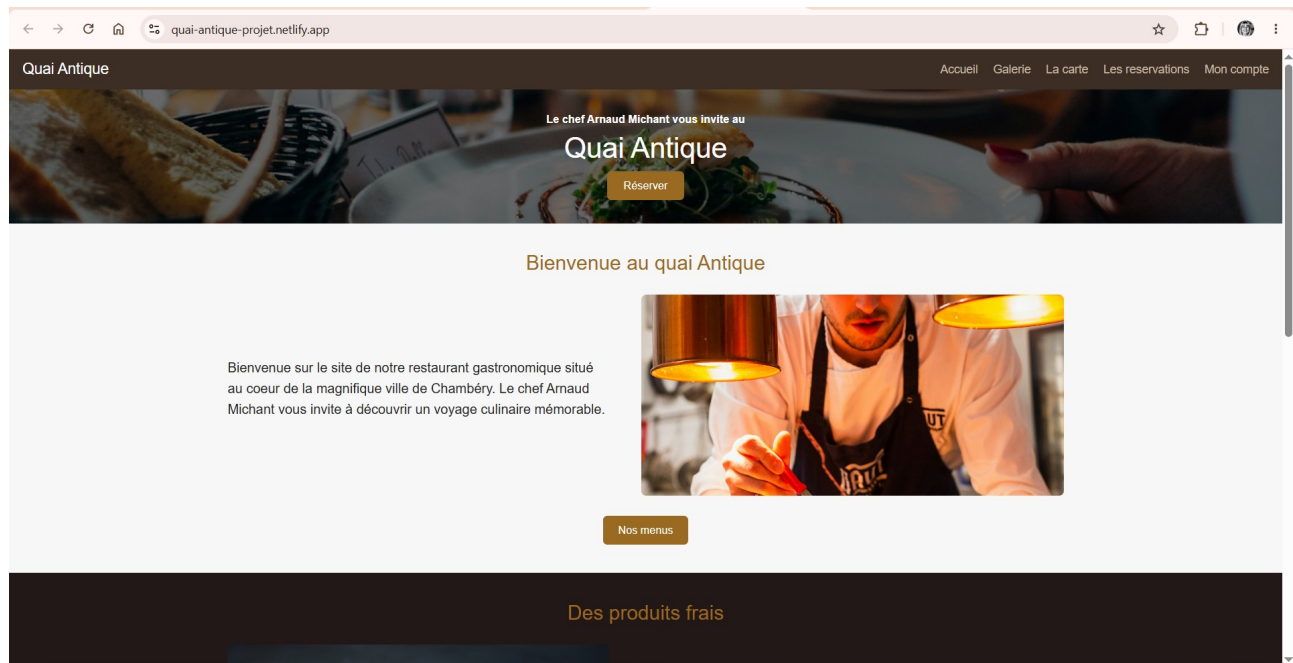


PROJET QUAI ANTIQUE — 1 PRÉSENTATION

Développement par : Larisa Faessel — Mai 2026

Projet Quai Antique



1. Présentation du projet

La Vision de Quai Antique

En tant qu'étudiante, je présente le projet Quai Antique. Il s'agit d'une application web moderne conçue pour un restaurant gastronomique haut de gamme.

* Description courte :

Une application web inspirée d'un restaurant de prestige permettant aux clients de découvrir l'établissement, de parcourir la carte de manière interactive et d'effectuer des réservations de tables en temps réel.

* Objectifs principaux :

Offrir une interface utilisateur haut de gamme (responsive et dynamique) couplée à un backend robuste pour la gestion sécurisée et fluide des demandes de réservation.

* Contexte de première remise :

Le projet est actuellement en cours de développement actif. La base technique est entièrement posée pour permettre de futures extensions rapides (authentification utilisateur complète et panneau d'administration).

2. Cahier des charges

Besoins & Fonctionnalités

* BESOIN CLIENT :

Disposer d'un site vitrine élégant et immersif permettant de consulter les plats, la galerie du restaurant et de faire une réservation de table en direct.

* FONCTIONNALITÉS INTERFACE (FRONTEND) :

- Page d'accueil : Présentation esthétique et bouton d'action d'appel à réservation.
- La Carte / Menu : Filtres interactifs par catégories (entrées, plats, desserts).
- Galerie photos : Grille adaptative pour afficher des photos en haute définition.
- Connexion / Inscription : Maquette de formulaires d'authentification.
- Formulaire de réservation : Saisie de la date, l'heure, du nombre de convives et des allergies.

* FONCTIONNALITÉS LOGIQUE (BACKEND & DB) :

- API REST RESTful : Serveur Express autonome pour intercepter les requêtes du frontend.
- Validation rigoureuse : Traitement et contrôle de conformité des données soumises côté serveur.
- Table 'reservations' PostgreSQL : Sauvegarde pérenne des réservations de table (identité, contact, allergies).

* CONTRAINTES TECHNIQUES :

- Adaptation fluide sur mobile, tablette et ordinateur de bureau (Responsive).
- Code modulaire structuré pour favoriser l'évolutivité (Architecture propre).
- Historique de commits régulier et propre (Versionnement Git).

3. Technologies utilisées

Le Stack Technique

Pour répondre aux exigences modernes du web, j'ai sélectionné et mis en œuvre des technologies stables et standardisées. L'accent a été mis sur la mise en place d'une architecture client-serveur découplée afin de préparer au mieux la transition vers de futurs frameworks.

* Frontend / Client : HTML5, CSS3, JavaScript (ES6+)

Rôle : Structure sémantique, styles réactifs (CSS Grid/Flexbox) et requêtes asynchrones Fetch API.

* Backend / API : Node.js, Express, CORS

Rôle : Serveur HTTP structuré, routage REST, middleware de sécurité CORS pour l'intégration.

* Base de Données : PostgreSQL (Supabase Cloud), pg

Rôle : Stockage relationnel, intégrité référentielle, connexion par pool robuste pour la performance.

* DevOps & Outils : Git / GitHub, Docker (Nginx), VS Code

Rôle : Contrôle des versions, conteneurisation pour le serveur statique local, environnement homogène.

4. Avancement du projet

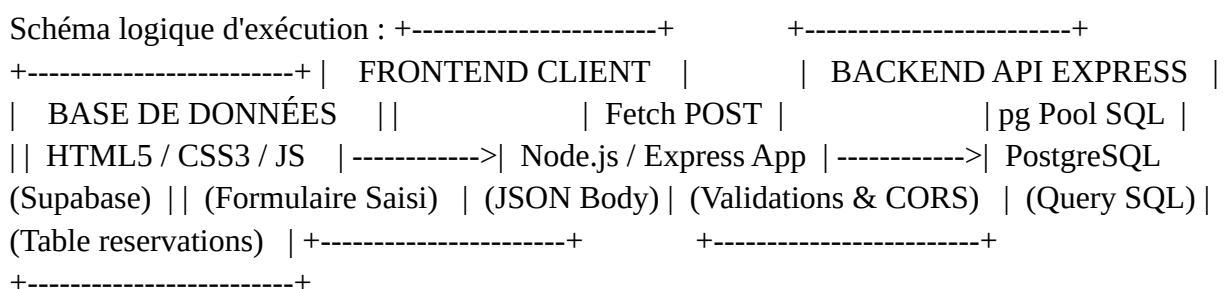
Suivi des Fonctionnalités

- * Page d'accueil & Vitrine : Page structurée avec hero-bannière styles groupés.
- * La Galerie des Plats : Grille responsive avec photos HD chargées localement.
- * La Carte & Menus : Onglets interactifs dynamiques animés en JavaScript (app.js).
Statut : REUSSI & VALIDE
- * Formulaire de réservation : Formulaire utilisateur avec validation locale et retour visuel.
- * API Backend Express : Routage REST (/api/reservations), routage de santé (/api/health).
- * Connexion PostgreSQL : Base de données Supabase connectée, script SQL de création de table.
- * Liaison Formulaire -> DB : Envoi des données POST JSON du Frontend vers le Backend Express.
- * Comptes & Espace Admin : Maquette existante. Sécurisation bcrypt, authentification JWT et panel admin.

5. Architecture technique

Flux de communication Client-Serveur

L'application implémente une architecture découpée en couches de type Client-Serveur. La communication entre le Frontend et le Backend est asynchrone et repose sur le format standard JSON.



Fonctionnement détaillé de l'intégration :

- Saisie : L'utilisateur remplit le formulaire de réservation sur l'interface (Date, heure, invités, allergies).
- Requête : Le script JavaScript (app.js) intercepte la soumission, valide les types de données et effectue une requête Fetch API en POST JSON vers le port 3003.
- Traitement : Le serveur Express intercepte la requête, applique les validations côté serveur (ex: nombre de convives > 0) et transmet les données au repository.

- Persistance : Le repository exécute une requête préparée via un pool de connexions (pg) vers PostgreSQL (Supabase) pour y insérer les informations de manière sécurisée.

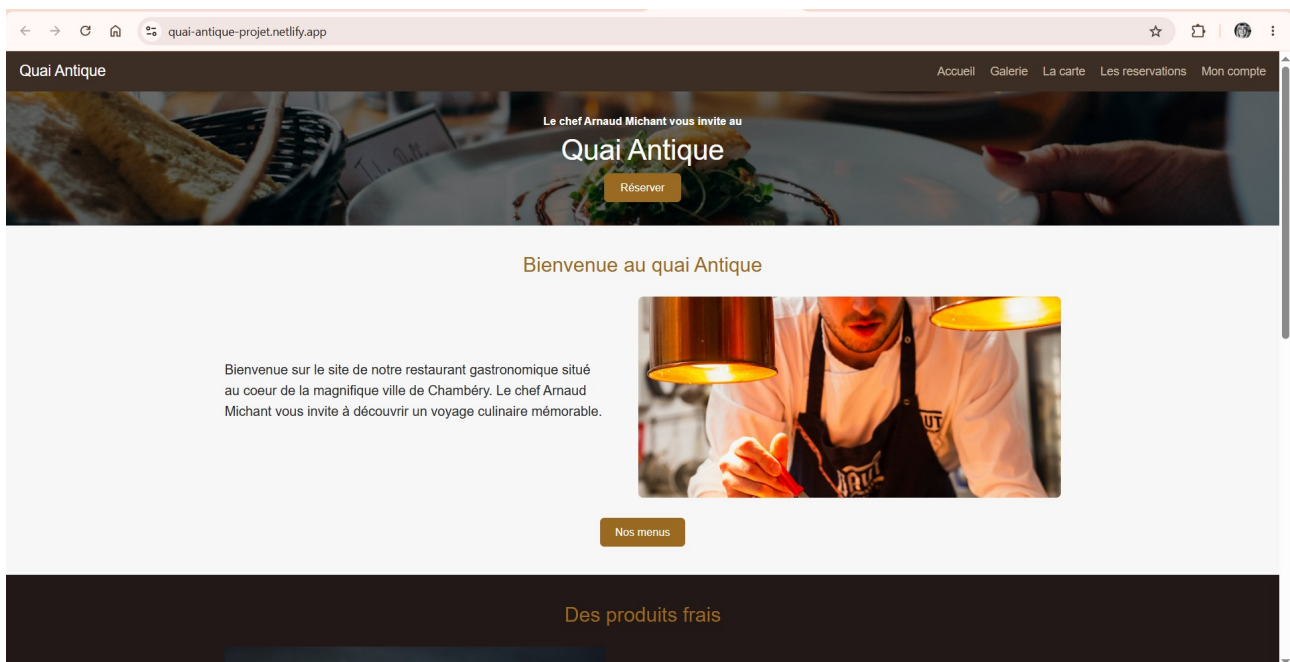
6. Captures d'écran : Interface Frontend

Aperçu des Écrans Principaux

L'interface utilisateur a été conçue pour refléter le standing d'un restaurant gastronomique. Les couleurs et polices ont été choisies avec soin (combinaison de tons sombres et dorés). Le site s'adapte de manière totalement automatique sur smartphone, tablette et écran large.

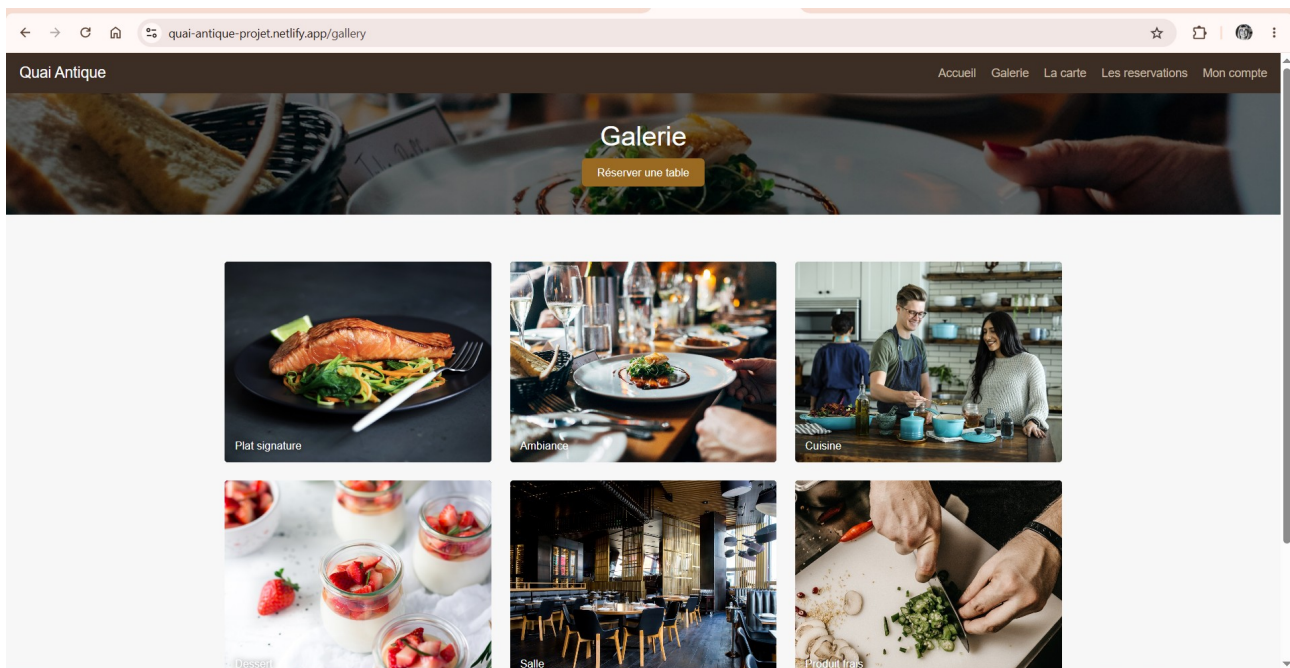
* Page d'accueil :

Bannière de présentation



* La Galerie photos :

Présentation responsive des créations culinaires.



* Formulaire de réservation :

Interface simple et intuitive de réservation de table.

6. Architecture de code : Contrôleur Backend & DB

Règles Métier & Intégrité

Afin de garantir un code maintenable et réutilisable, le backend Express est organisé selon le patron Repository :

- `controllers/reservation.controller.js` : Gère le traitement de la requête HTTP, extrait les paramètres de réservation, valide que tous les champs obligatoires sont bien remplis, applique des règles métier (ex: vérifier que le nombre de personnes est bien un entier strictement positif) et renvoie la réponse correspondante (201 Created ou codes d'erreur 400, 500, 503).
- `repositories/reservation.repository.js` : Se charge exclusivement des interactions SQL avec PostgreSQL via des requêtes paramétrées sécurisées prévenant les injections SQL.
- `config/database.js` & `initDatabase.js` : Gère la connexion sécurisée par pool pg et crée automatiquement la table `reservations` au lancement du serveur avec reconnexion automatique en cas d'indisponibilité.

Extrait de code du contrôleur de réservation :

```
// controllers/reservation.controller.js async function createReservation(req, res) { const
{ firstName, lastName, email, date, time, guests, allergies } = req.body;

if (!firstName || !lastName || !email || !date || !time || !guests) { return res.status(400).json({ message:
'Tous les champs obligatoires doivent etre remplis' }); }

const guestCount = Number(guests); if (!Number.isInteger(guestCount) || guestCount <= 0) { return
res.status(400).json({ message: 'Le nombre de personnes doit etre un nombre entier positif' }); }

try { const reservation = await reservationRepository.createReservation({ firstName, lastName,
email, date, time, guests: guestCount, allergies }); return res.status(201).json({ message:
'Reservation enregistree', reservation }); } catch (error) { if
(error.message.includes('DATABASE_URL')) { return res.status(503).json({ message: 'Base de
donnees indisponible' }); } return res.status(500).json({ message: 'Erreur lors de l
enregistrement' }); } }
```

7. Gestion des versions : Git & GitHub

Méthodologie de Versionnement

Le suivi des versions est un pilier de mon apprentissage. Dès le début du projet, j'ai mis en place un dépôt Git local et je l'ai connecté à mon compte GitHub afin de sauvegarder mon travail et de le rendre disponible pour la validation pédagogique.

- Organisation des commits : Chaque commit correspond à une tâche claire et unitaire de développement. Les messages de commit respectent les conventions afin de faciliter la lecture de l'historique.
- Sécurisation du code : J'ai veillé à mettre en place un fichier `.gitignore` robuste pour empêcher la fuite de secrets ou de dépendances lourdes (exclusion de `node_modules`, des fichiers `.env` de configuration locale contenant les identifiants de la base de données).
- Dépôt GitHub : https://github.com/catcodecat/Quai_Antique_Project
- Site Vitrine Déployé : <https://quai-antique-projet.netlify.app/>

8. Difficultés rencontrées et solutions

Voici les principales difficultés que j'ai rencontrées et comment j'ai réussi à les surmonter :

* La gestion de la politique de sécurité CORS :

- Difficulté : Lors des premiers tests de soumission du formulaire de réservation depuis mon frontend statique, les navigateurs bloquaient les requêtes à cause des règles de sécurité CORS (Cross-Origin Resource Sharing).

- Solution : J'ai surmonté cela en configurant correctement le middleware cors côté Express pour autoriser spécifiquement mon origine locale de développement (<http://localhost:8080>).

* L'intégration et la résilience de la base de données :

- Difficulté : Établir la connexion avec PostgreSQL (Supabase) a nécessité de bien appréhender l'asynchronisme de JavaScript. Si la base de données mettait du temps à s'éveiller au démarrage (mode veille cloud), le serveur Express plantait.

- Solution : J'ai implémenté un système robuste de reconnexion automatique (jusqu'à 5 tentatives avec intervalle d'attente asynchrone) dans mon script d'initialisation de base de données `initDatabase.js`.

* La validation rigoureuse des données :

- Difficulté : Assurer que seules des réservations valides et saines soient stockées en base (sécurité de type et cohérence des valeurs).

- Solution : J'ai mis en place une double couche de validation : dans le formulaire frontend HTML5 et dans le contrôleur backend Express (contrôle de type et de valeurs numériques), assurant la qualité et la sécurité de la donnée.

9. Prochaines étapes du développement

Plan de Travail pour l'Étape 2

* SÉCURISATION ET AUTHENTIFICATION DES COMPTES CLIENTS :

- Implémenter le hachage sécurisé des mots de passe dans la base de données avec la bibliothèque `bcrypt` côté backend.

- Générer et valider des jetons sécurisés JWT (JSON Web Tokens) lors de la connexion pour maintenir la session de l'utilisateur.

- Protéger les routes sensibles de l'API REST grâce à des middlewares d'authentification personnalisés.

*** DÉVELOPPEMENT DU PANNEAU D'ADMINISTRATION COMPLET :**

- Créer des écrans d'administration permettant de modifier dynamiquement les plats de la carte, les prix et les descriptions.
- Gérer les jours et horaires d'ouverture du restaurant de manière interactive.
- Pouvoir consulter, filtrer et valider/annuler les demandes de réservations des clients.

*** REFACTORISATION ET MIGRATION COMPOSANTS REACT + VITE :**

- Refactoriser le frontend statique HTML/CSS/JS actuel dans un projet moderne React + Vite pour un développement plus fluide.
- Découper l'interface utilisateur en composants autonomes et réutilisables (ex: Header, Footer, MenuTabs, ReservationForm).

